

An experimental study of a reliable IoT gateway

Byungseok Kang^a, Hyunseung Choo^{b,*}

^a Department of Data Science, Sejong University, Seoul, 05006, Republic of Korea

^b Department of Computer Science and Engineering, Sungkyunkwan University, Suwon, 440-746, Republic of Korea

Received 13 September 2016; received in revised form 6 March 2017; accepted 15 April 2017

Available online 27 April 2017

Abstract

The Internet of Things (IoT) will include new devices specifically designed for IoT compatibility and systems that are already in place today and operate outside of IoT networks. However, the path to creating cloud networks of interconnected devices requires a means for devices that are not IP-based to connect without having to bear the cost of a full Ethernet or WiFi interface with the accompanying protocol stack. This can be achieved through the use of IoT gateways that bridge these devices to the Internet in the context of real-world applications. In this paper, we introduce an experimental study of a reliable and self-configurable IoT gateway that is developed in a laboratory testbed by using the IoTivity framework.

© 2017 The Korean Institute of Communications and Information Sciences (KICS). Publishing Services by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Keywords: IoT; IoT gateway; Smart gateway; Reliability

1. Introduction

Many works have been carried out to advance Internet of Things (IoT) technology in business domains [1,2] because of the value gained by connecting an endpoint device for system reliability, automation, and centralized management. Although developed for business applications, many of these advances are applicable to all types of smart systems, including wearables, mobile phones, healthcare equipment, security devices, and cloud and deep-learning systems [3,4]. For engineers, the greatest challenge in designing the IoT is reliability [5,6]. The implementation of robust and secure access to the Internet or a wide area network (WAN) is outside their range of experience. To make design even more difficult, developers need to support access to multiple devices that are limited in their processing capability. Reliability must also be added in a way that does not adversely impact the overall cost efficiency [7–9].

The diversity of endpoints that a gateway must support raises design concerns [10,11] as well. The direct connection of a

simple device such as a humidity sensor to the Internet can be complex and expensive, especially if the device does not have its own processor. In addition, different types of end equipment support varying interfaces. The collection and aggregation of data from a disparate set of nodes require a means for bridging devices with a range of processing capabilities and interfaces together in a consistent and reliable way [12,13]. Gateways offer an elegant means for simplifying the networking of “things”. They achieve this by supporting the different methods of device connection, whether this is a varying voltage from a raw sensor, a stream of data over I2C from an encoder, or periodic updates from an appliance via WiFi. Gateways effectively mitigate the great variety and diversity of devices by consolidating data from disparate sources and interfaces and bridging them to the Internet. The result is that individual nodes do not need to bear the complexity or cost of a high-speed Internet interface in order to be connected.

An IoT gateway is an intelligent component based on an IoT platform [14,15]. It is typically employed between the network of the machine-to-machine (M2M) devices and the remote peers (e.g., clients) over the internet. The basic goal of the gateway is to settle the heterogeneity between different endpoint networks and the Internet, strengthen the management of the endpoint networks, and bridge the traditional Internet with

* Corresponding author.

E-mail addresses: bskang@sejong.ac.kr (B. Kang), choo@skku.edu (H. Choo).

Peer review under responsibility of The Korean Institute of Communications and Information Sciences (KICS).

endpoint networks. As state-of-the-art IoT gateways [16,17] are designed to connect to mobile and wireless networks, they emphasize the provision of flexible connections among smart devices and a user's cloud to enable intelligent big data analysis and data-driven decision-making. Furthermore, they can be flexibly configured with different protocol-ready modules to communicate with end nodes or I/O devices.

However, current IoT gateways [14] operate passively or semi-automatically. This means that, when a user buys a new IoT device, he/she manually installs the device using the setup manual. After that, the IoT gateway asks the user whether a new device should be registered. To solve this problem, we propose a reliable IoT gateway for small-scale IoT environments in this paper. When a user brings a new device in their home, the IoT gateway automatically detects this new device and registers this device by itself. If the user discards old devices from his/her home, the smart IoT gateway automatically deletes that device from the device list. In our system, the user does not have to worry about registering and installing new IoT devices. In the following sections, we detail our proposed self-configurable IoT gateway. The development of the testbed is presented in Section 3. Finally, we conclude the paper in Section 4.

2. Basic design of the self-configurable IoT gateway

IoT gateways available today allow a user to configure a few properties such as the TV channel, light, refrigerator, etc. However, future smart homes, particularly those based on software-defined networks, are likely to have very rich configuration controls. The goal of this research is to develop algorithms that enable an IoT gateway to automatically configure itself. We propose an initial study of autoconfiguration that considers two properties: dynamic discovery and autoregistration.

We designed the IoT gateway using the IoTivity framework [18]. The interaction model of IoTivity is similar to the client/server model of HTTP and uses the CoAP protocol for device-to-device (D2D) communications. We use the options and payload fields to collect each device's attribute information. The detailed information of the CoAP fields in the header is defined in an IETF RFC document [19]. However, D2D interactions typically result in an IoTivity implementation acting in both client and server roles. An IoTivity operation is equivalent to that of HTTP and is sent by a client to request an action on a resource (identified by a URI) on a server. The server then sends a response with a response code; this response includes a resource representation. IoTivity makes use of the GET, PUT, POST, DELETE, and OBSERVE operations in a similar manner to HTTP with semantics. We modified the existing "OBSERVE" operation and created a new operation "INIT". The details of each operation are described as follows:

GET retrieves a representation for the information that currently corresponds to the resource identified by the request URI. If the request includes an accept option, this indicates the preferred content format of a response. If the request includes an ETag Option, this method requests that ETag be validated

and that the representation be transferred only if validation failed.

POST requests that the representation enclosed in the request be processed. The actual function performed by this method is determined by the origin server and dependent on the target device. It usually results in a new IoT device being created or the target device being updated.

PUT requests that the resource identified by the request URI be updated or created with the enclosed representation. The representation format is specified by the media type and content coding given in the option of the content format, if provided.

DELETE requests that the resource identified by the request URI be deleted. A 2.02 (deleted) response code should be used on success or in case where the resource did not exist before the request.

OBSERVE fetches and registers as an observer for the value of an IoT device. The handling of observation registration is application-specific. It should not be assumed that a device is observable or that a device can handle any specific number of observers. If the master server responds with a success (2.12) code, the registration is considered successful.

INIT brings a device and its attribute information to the master server. Normally, this operation is generated by a sensor or an IoT machine and sends this information to the IoT gateway. If the IoT gateway receives this message, the gateway updates its device and attribute table based on the received data.

In Fig. 1(a), the current system detects a new IoT device by using the OBSERVE operation, but it does not know its detailed attributes and functions. Therefore, the master server sends a "FAIL" message to the slave server. However, in Fig. 1(b), the IoT machine sends its attribute information through the "INIT" operation. After receiving this INIT message, the IoT gateway automatically registers the attributes to the device and attribute table.

3. Testbed implementation and measurement

In the following section, we present the development of an in-home-scale testbed for experimentation and evaluation of the proposed IoT gateway. We concentrate the measurement on the device setup time.

3.1. Testbed development

The proposed testbed includes three main types of devices, which are the master server in Fig. 2(a), the two slave servers (client devices) in Fig. 2(b), and a user's mobile device (see Fig. 3). The three server devices in the proposed testbed are built on a standard Raspberry-Pi-embedded machine and use a general Linux-kernel-based operating system called Raspbian. Thus, all devices can be easily reconfigured to evaluate various network environments. Raspberry Pi is widely used to implement IoT-based smart environments [20–22]. We installed

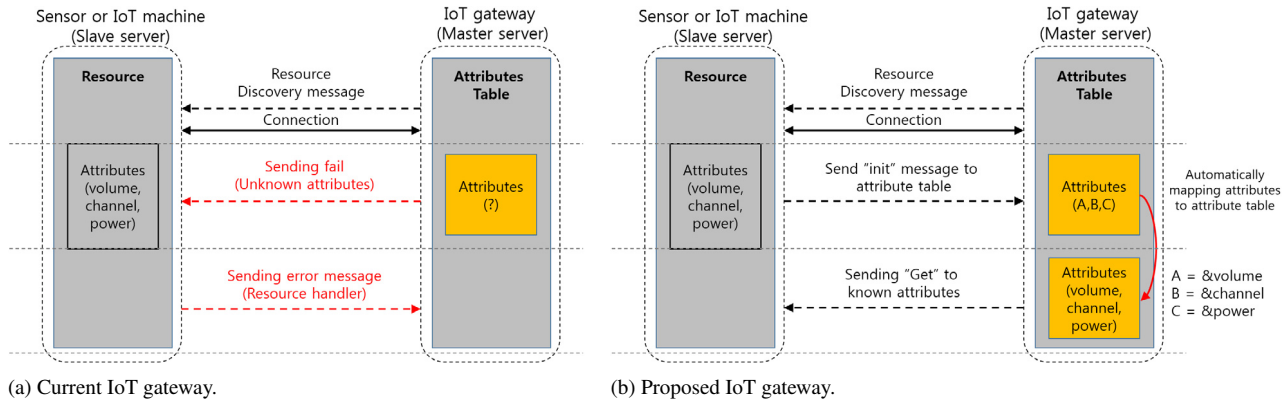
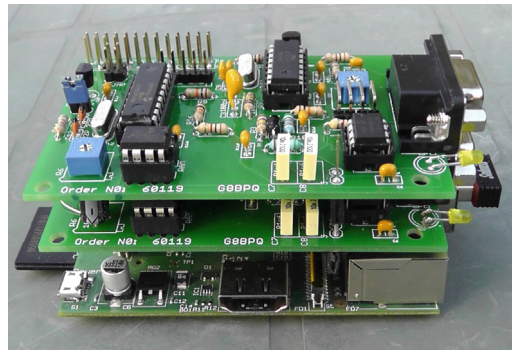


Fig. 1. Device discovery and auto-registration.



(a) Master server.



(b) Slave server (client).

Fig. 2. Laboratory-scale IoT testbed.



Fig. 3. Example of device control of an IoT lamp.

Android 6.0 (code name Marshmallow) and the IoTivity platform [18] for easy-to-manage wireless IoT devices. IoTivity is an open-source-software framework enabling reliable D2D communication to address the emerging needs of the IoT. On the user's mobile device, we installed the Android Marshmallow operating system.

A network substrate considers the interconnections between the master and slave servers, as shown in Fig. 2. If the master server is connected with IoTivity, then IoTivity identifies which machine is matched. If IoTivity cannot find any entry, then the master server sends an OBSERVE message to the slave servers on the basis of the IoT communication policy. For the testbed measurements, we use the recently published IoTivity framework (version 1.1.1, July 2016). We also use both the

WiFi and LTE interfaces to communicate with the user's mobile device. Fig. 3 shows result for the attribute registration of an IoT lamp. In this figure, the user can control the attribute functions of the lamp (on/off).

3.2. System measurement

In order to carry out a performance evaluation of the proposed gateway, we measured each device's registration time in the wireless network environments (WiFi and LTE). We increased the number of slave devices after every measurement. We also increased the number of measurements up to 100. In Fig. 4, the results show that the proposed gateway registers each device within 2.4 s. If this system measurement is carried out under the wired-network condition, the registration time will be improved further.

4. Conclusion

In this paper, we implemented a gateway that is better known as a reliable and self-configurable IoT gateway and designed to be suitable for in-home-scale environments. For testbed development, we use the state-of-the-art IoTivity framework. We have a plan for the system measurements by checking the performance of the proposed gateway. In addition, our future work will extend the reliable IoT gateway to large-scale and energy-aware mobile environments.

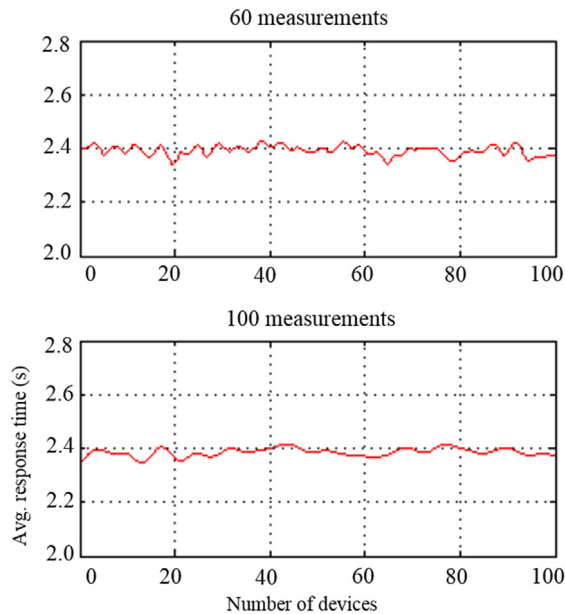


Fig. 4. Average response time for attribute registration.

Acknowledgments

This work was supported by the G-ITRC Program under Grant IITP-2015R6812-15-0001, the NRF Research Fellow program under Grant NRF-2016R1A6A3A11934080, and the NRF Korea under Grant 2010-0020210.

Conflict of interests

The authors declare that there is no conflict of interest in this paper.

References

- [1] S. Leminen, M. Westerlund, M. Rajahonka, R. Siuruainen, Towards iot ecosystems and business models, in: *Internet of Things, Smart Spaces, and Next Generation Networking*, Springer, 2012, pp. 15–26.
- [2] Q. Xiacong, Z. Jidong, Study on the structure of internet of things (iot) business operation support platform, in: *2010 12th IEEE International Conference on Communication Technology, (ICCT)*, IEEE, 2010, pp. 1068–1071.
- [3] B. Kang, H. Choo, An sdn-enhanced load-balancing technique in the cloud system, *J. Supercomput.* (2016) 1–24.
- [4] B. Kang, H. Choo, A deep-learning-based emergency alert system, *ICT Express* 2 (2) (2016) 67–70.
- [5] W. Li, Y. Miao, Y.-w. Tang, D. Liu, B. Hu, Iot system reliability testing and evaluate technology, *Software* 4 (2012) 001.
- [6] L. Li, Z. Jin, G. Li, L. Zheng, Q. Wei, Modeling and analyzing the reliability and cost of service composition in the iot: A probabilistic approach, in: *2012 IEEE 19th International Conference on Web Services, (ICWS)*, IEEE, 2012, pp. 584–591.
- [7] B. Kang, Network-based job dispatching in the cloud, in: *Information Sciences and Systems 2015*, Springer, 2016, pp. 233–240.
- [8] B. Kang, H. Choo, A cluster-based decentralized job dispatching for the large-scale cloud, *EURASIP J. Wirel. Comm. Netw.* 2016 (1) (2016) 1–8.
- [9] B. Kang, P. Nguyen, V. Zalyubouskiy, H. Choo, A distributed delay-efficient data aggregation scheduling for duty-cycled wsns, *IEEE Sens. J.* 17 (2017).
- [10] S. Guoqiang, C. Yanming, Z. Chao, Z. Yanxu, Design and implementation of a smart iot gateway, in: *Green Computing and Communications (GreenCom), 2013 IEEE and Internet of Things (iThings/CPSCoM), IEEE International Conference on and IEEE Cyber, Physical and Social Computing*, IEEE, 2013, pp. 720–723.
- [11] X. Jiang, D. Li, S. Nie, J. Luo, Z. Lu, An enhanced iot gateway in a broadcast system, in: *2012 9th International Conference on Ubiquitous Intelligence & Computing and 9th International Conference on Autonomic & Trusted Computing, (UIC/ATC)*, IEEE, 2012, pp. 746–751.
- [12] B. Kang, N. Kwon, H. Choo, Developing route optimization-based pmipv6 testbed for reliable packet transmission, *IEEE Access* 4 (2016) 1039–1049.
- [13] B. Kang, S. Myoung, H. Choo, Distributed degree-based link scheduling for collision avoidance in wireless sensor networks, *IEEE Access* 4 (2016) 7452–7468.
- [14] G. Yang, L. Xie, M. Mäntysalo, X. Zhou, Z. Pang, L. Da Xu, S. Kao-Walter, Q. Chen, L.-R. Zheng, A health-iot platform based on the integration of intelligent packaging, unobtrusive bio-sensor, and intelligent medicine box, *IEEE Trans. Ind. Inf.* 10 (4) (2014) 2180–2191.
- [15] R. Lea, M. Blackstock, City hub: A cloud-based iot platform for smart cities, in: *2014 IEEE 6th International Conference on Cloud Computing Technology and Science, (CloudCom)*, IEEE, 2014, pp. 799–804.
- [16] L. Xing, B. Zhang, Q. Wu, L. Xie, Research of an embedded iot gateway applied to post-earthquake rescue, *Int. J. Online Eng.* 11 (2015).
- [17] S.K. Datta, C. Bonnet, N. Nikaein, An iot gateway centric architecture to provide novel m2m services, in: *2014 IEEE World Forum on Internet of Things, (WF-IoT)*, IEEE, 2014, pp. 514–519.
- [18] L. Foundation, LINUX FOUNDATION IoTivity, 2016. URL <https://www.iotivity.org/>.
- [19] Z. Shelby, K. Hartke, C. Bormann, The constrained application protocol (coap), Tech. Rep., 2014.
- [20] A.N. Ansari, M. Sedky, N. Sharma, A. Tyagi, An Internet of things approach for motion detection using raspberry pi, in: *2014 International Conference on Intelligent Computing and Internet of Things, (ICIT)*, IEEE, 2015, pp. 131–134.
- [21] E.A.D. Ballesteros, C.E.C. Calderon, Y.C. Calderon, E.G. Strauss, Android management tool, that controls electronic devices through a Raspberry Pi under the IoT model, in: *Computing Colombian Conference (10CCC), 2015 10th, IEEE*, 2015, pp. 237–244.
- [22] S. Jain, A. Vaibhav, L. Goyal, Raspberry pi based interactive home automation system through e-mail, in: *2014 International Conference on Optimization, Reliability, and Information Technology, (ICROIT)*, IEEE, 2014, pp. 277–280.